

STREAMSENSE

Model & Preprocessing Interface Contract (MPIC) v1.0

Optimized Solutions Limited (OSL) — OSL-IPL-2026-INT-002 — FROZEN SPECIFICATION

This document defines the frozen interface contract between the preprocessing pipeline (Track A, Python/PyTorch) and the model inference / streaming runtime (Track B, C++ and MATLAB validation). Both Track A and Track B implementations must conform exactly to the parameters below. No changes are permitted to this document without joint sign-off from both tracks.

1. Signal & Feature Extraction Parameters

Field	Value	Notes
Sample rate	16000 Hz	Input must be 16 kHz mono float32
Frame length	16000 samples	1 second; pad with zeros or crop
FFT size (n_fft)	512	
Hop length	160 samples	10 ms at 16 kHz
Mel bins (n_mels)	64	
Window	Hann (periodic)	
Center	False	Critical — gives T = 97 output frames
Power	2.0	Power spectrogram (not magnitude)
Log scaling	$10 * \log_{10}(\text{mel} + 1e-10)$	epsilon = 1e-10
Clip floor	-80 dB	<code>torch.clamp(min=-80.0)</code>
Output mel shape	[64, 97]	[n_mels, time_frames]

2. Global Normalization Statistics

Computed once over the full training split (26,984 files) via `compute_normstats.py`. Frozen — both implementations must apply: $(\text{mel_db} - \text{global_mean}) / \text{global_std}$.

Field	Value
global_mean	-30.785545 dB
global_std	22.157099 dB

3. Model Input / Output Tensor Contract

Field	Value	Notes
Input tensor name	"input"	
Input shape	[1, 1, 64, 97]	[batch, channel, n_mels, time]
Input dtype	float32	
Output tensor name	"logits"	

Output shape	[1, 10]	10-class logits
Output dtype	float32	Same for FP32 and INT8 models
Output scale / zero-point	N/A	QDQ INT8 model has float32 I/O, identical to FP32
Model format	ONNX	FP32 (export_onnx.ipynb) and QDQ INT8 (quantize_ptq.ipynb)

4. Output Class Label Order

Output logit index corresponds to class label as follows (from `class_labels.json`):

Index	0	1	2	3	4	5	6	7	8	9
Label	yes	no	up	down	left	right	on	off	stop	go

5. Numeric Tolerance (Golden Vector Parity)

Comparison	Max Abs Error (tolerance)
Same implementation (Python vs Python)	1e-4
Cross implementation (Python vs C++ / MATLAB)	5e-4
Manifest tolerance_max_abs_error	0.0005

Pass criteria: top-1 predicted class must match exactly between implementations; per-element absolute error on the normalized feature tensor must not exceed the cross-implementation tolerance above.

6. Reference Implementation

The canonical implementation of this contract is `mel_pipeline.py` (function `preprocess(samples) -> torch.Tensor[1, 1, 64, 97]`), located in `training/` and included in the deployment package under `preprocessing/`. Any cross-implementation (C++, MATLAB) must reproduce the 9-step pipeline defined therein and validate against the 10 golden vectors in `golden_vectors/` (manifest: `golden_vectors/manifest.json`).

Status: FROZEN. Sign-off required from both Track A and Track B before implementation proceeds against this contract. Any required change must go through a new version (v1.1) with joint review.

Track A sign-off: _____ Date: _____

Track B sign-off: _____ Date: _____